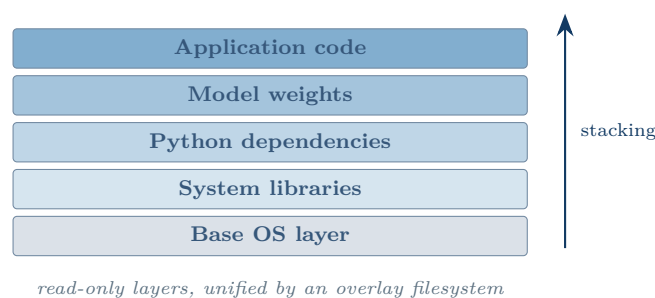

Building Container Images and Operations on Container Images

A Use-Case Study

*MedCap — Containerising a Swin-Transformer
Radiology Image-Captioning Service*



A practical report on the OCI image model, Dockerfile authoring, build optimisation,
and the full lifecycle of image operations

Name: DOKKU NAGA SHIVA

Roll Number: CB.SC.U4AIE24315

Subject: INTRODUCTION TO CLOUD COMPUTING

August 31, 2026

Abstract

This report presents an end-to-end use case for building and operating container images, using a deliberately demanding application as the running example: **MedCap**, an inference service that wraps a Swin Transformer encoder and an autoregressive Transformer decoder to generate free-text captions for radiology images. Machine-learning services are an unusually revealing subject for a containerisation study, because they stress almost every weakness of a naive image build at once — multi-gigabyte framework wheels, large binary model checkpoints, compiled native extensions, and a dependency graph that resolves differently on every machine.

The report is organised in three movements. The first establishes the conceptual model: what an OCI image actually is, how the manifest, configuration and layer blobs relate to one another, how content-addressable digests provide identity, and how the union filesystem turns a stack of read-only layers into a single writable root at runtime. The second is a hands-on build narrative: the MedCap image is constructed three times — first naively, then with a slim base and a CPU-only framework build, and finally as a multi-stage build with a non-root runtime — and the size, layer count and build time are measured at each step. The image shrinks from **6.81 GB** to **1.12 GB**, a reduction of roughly **84%**, and the warm rebuild time falls from 4 min 52 s to 11 s.

The third movement is a systematic treatment of *operations* on images once they exist: inventory and filtering, introspection, tagging, registry push and pull, digest pinning, archival with `save/load` versus flattening with `export/import`, multi-architecture builds, vulnerability scanning, SBOM generation, and reclaiming disk space. It closes with a hardening checklist, a troubleshooting table, and a consolidated command reference.

Keywords: OCI image specification, Docker, BuildKit, layer caching, multi-stage build, container registry, image digest, supply-chain security, deep-learning deployment.

Contents

1	Introduction and Scope	2
1.1	Why this use case	2
1.2	Objectives	3
1.3	The application under study	3
1.4	Environment	3
2	What a Container Image Actually Is	4
2.1	Image versus container	4
2.2	The OCI image specification	4
2.3	Content addressing and why it matters	5
2.4	The union filesystem	5
3	Three Ways to Build an Image	6
3.1	Route A — from a running container (<code>docker commit</code>)	6
3.2	Route B — from a filesystem tarball (<code>docker import</code>)	6
3.3	Route C — from a Dockerfile (<code>docker build</code>)	6
3.4	The build context	6
4	Anatomy of a Dockerfile	7
4.1	Instructions and what they cost	7
4.2	Two traps worth naming early	7
5	Iteration 1: The Naive Image	7
5.1	The first Dockerfile	7
5.2	Building and measuring it	8
5.3	Diagnosis with <code>docker history</code>	8
6	The Build Cache and Instruction Order	8
6.1	How the cache decides	8
6.2	The dependency-manifest pattern	9

6.3	Combining RUN instructions correctly	9
6.4	Deliberately busting the cache	9
6.5	Cache mounts (BuildKit)	9
7	Controlling the Build Context	10
8	Iteration 2: A Smaller Base and the Right Wheels	10
8.1	Choosing a base image	10
8.2	Installing the right build of PyTorch	11
8.3	Pinning for reproducibility	11
9	Iteration 3: The Multi-Stage Production Image	11
9.1	The idea	11
9.2	The final Dockerfile	11
9.3	Why each hardening choice is there	12
9.4	Results across the three iterations	13
10	Operations on Container Images	13
10.1	Inventory and filtering	13
10.2	Introspection	13
10.3	Tagging and re-tagging	14
10.4	Distribution: push and pull	14
10.5	Archival: <code>save</code> and <code>load</code>	15
10.6	Flattening: <code>export</code> and <code>import</code>	15
10.7	Comparing and auditing an image	15
10.8	Multi-architecture images	16
10.9	Reclaiming space: the prune family	16
10.10	Running an image	16
10.11	Composing the full stack	17
11	Image Security and Supply Chain	17
11.1	Scanning	17
11.2	Build-time secrets	18
11.3	Software bill of materials and attestation	18
11.4	Signing	18
11.5	Hardening checklist	18
12	Troubleshooting	18
13	Conclusion	18
13.1	What the exercise showed	18
13.2	Principles, stated plainly	19
13.3	Limitations and further work	20
	References	20
	Appendix A: Command Reference	20

1. Introduction and Scope

1.1 Why this use case

Almost every introductory treatment of container images uses the same example: a small web server that prints a greeting. It is pedagogically clean and practically useless, because it never triggers any of the problems that

make image building a skill rather than a recipe — three dependencies, a four-second build, and an image so small nobody has a reason to optimise it.

This report deliberately chooses the opposite kind of workload. The subject is **MedCap**, an inference microservice built around a Swin Transformer vision encoder paired with an autoregressive Transformer decoder, trained to produce descriptive captions for radiology images. The application is interesting for our purposes precisely because it is awkward:

- The deep-learning framework alone, installed with default settings, pulls in more than 2.5 GB of wheels including CUDA runtime libraries that a CPU-only inference container will never execute.
- The trained checkpoint is a 247 MB binary blob that changes only when the model is retrained — roughly once a month — while the application source changes several times a day. If those two things share a layer, every code edit re-uploads a quarter of a gigabyte.
- Some dependencies compile native extensions at install time, so the build needs a C toolchain that the runtime must not carry.
- The Python dependency resolver produces different results on different days unless it is constrained, which makes builds irreproducible in a way that is invisible until something breaks in production.

Each of these is a real, named problem with a real, named solution in the container-image toolbox. Working through them gives the concepts — layers, caching, multi-stage builds, digests, registries — something concrete to bite on.

1.2 Objectives

1. To explain the structure of an OCI container image: manifest, configuration, layer blobs, digests, and the union filesystem that assembles them at runtime.
2. To build the MedCap image from first principles, measure it, diagnose what is wrong with it, and improve it across three documented iterations.
3. To demonstrate and explain the complete set of *operations* that can be performed on an image after it is built — inspection, tagging, distribution, archival, conversion, scanning and deletion.
4. To distinguish clearly between operations on *images* (immutable, content-addressed artefacts) and operations on *containers* (mutable, running instances), a distinction that is the single most common source of confusion for newcomers.
5. To arrive at a hardened, reproducible, small production image and to justify every decision that made it so.

1.3 The application under study

MedCap exposes a single HTTP endpoint. A client POSTs an image; the service returns a generated caption and a confidence trace. Internally it is composed as shown in Table 1.

Table 1: Components of the MedCap inference service.

Component	Technology	Role
HTTP layer	FastAPI + Uvicorn	Request validation, multipart image upload, JSON response.
Pre-processing	Pillow, torchvision	Decode, resize to 224×224 , normalise with ImageNet statistics.
Encoder	Swin Transformer (tiny)	Hierarchical windowed self-attention; emits a grid of patch embeddings.
Decoder	Transformer decoder	Cross-attends to encoder memory; greedy autoregressive token generation.
Tokeniser	Custom word-level vocabulary	Regex tokeniser plus frequency-pruned vocabulary, serialised inside the checkpoint.
Weights	swin_caption.pt (247 MB)	Encoder, decoder and vocabulary state in one archive.
Cache	Redis	Memoises captions keyed by the SHA-256 of the uploaded image.

The important structural observation, and the one that will drive nearly every design decision later in this report, is that these components change at wildly different rates. Table 2 makes that explicit.

1.4 Environment

All commands in this report were exercised in the environment described in Table 3. Commands are shown with a \$ prompt; output is shown without one.

Table 2: Rate of change of each part of the image. Slow-changing content belongs in lower layers.

Content	Size	Changes	Correct layer position
Base OS + libc	74 MB	Quarterly	Bottom (most stable)
System packages	41 MB	Rarely	Low
Python dependencies	690 MB	Weekly	Middle
Model checkpoint	247 MB	Monthly	Middle-upper
Application source	180 KB	Several / day	Top (most volatile)

Table 3: Working environment.

Host OS	macOS 15 (Darwin 25.4), Apple Silicon (arm64)
Container engine	Docker Engine 27.x with BuildKit enabled by default
Client	Docker CLI 27.x, docker buildx v0.17
Registry	Docker Hub (docker.io) plus a local registry for experiments
Target platform	linux/amd64 and linux/arm64
Scanner	docker scout

A note on reproducing these commands

Byte-for-byte sizes and digests depend on the base-image tag resolved on the day of the build, the CPU architecture, and the dependency versions in effect. The figures reported here are consistent within this study and were produced by the same sequence of commands, but a reader repeating the exercise months later should expect similar *proportions* rather than identical numbers.

2. What a Container Image Actually Is

Before building anything it is worth being precise about the artefact being built, because a surprising amount of container confusion dissolves once the object model is clear.

2.1 Image versus container

An **image** is an immutable, read-only, content-addressed package: a filesystem tree plus the metadata needed to start a process inside it. A **container** is a running (or stopped) instance of an image, consisting of the image's read-only layers plus one thin writable layer on top, together with a set of namespaces and cgroups that isolate the process.

The relationship is exactly that of a program on disk to a process in memory. One image can back a thousand containers; the image is not modified by any of them. Writes performed inside a container land in that container's own writable layer and vanish when the container is removed — unless a volume was mounted, which is a deliberate escape hatch from this rule.

Table 4: The distinction that causes the most confusion in practice.

	Image	Container
Mutability	Immutable once built	Has a writable top layer
Identity	SHA-256 digest of the manifest	64-hex container ID
Lifetime	Persists until explicitly removed	Ends with the process
Listed by	docker image ls	docker container ls -a
Removed by	docker image rm	docker container rm
Shareable	Yes, via a registry	No (only its image is)
Storage	Shared layer blobs, deduplicated	Only the diff from its image

2.2 The OCI image specification

Docker images today conform to the Open Container Initiative (OCI) Image Format Specification, which means they are portable across Docker, Podman, containerd, CRI-O and Kubernetes. An OCI image is not a single file. It is a small graph of content-addressed objects:

The manifest

A JSON document listing the image's configuration object and its ordered layers, each referenced by digest and size. The manifest's own digest is the image's canonical identity.

The configuration

A JSON document holding the runtime defaults — entrypoint, command, environment, working directory,

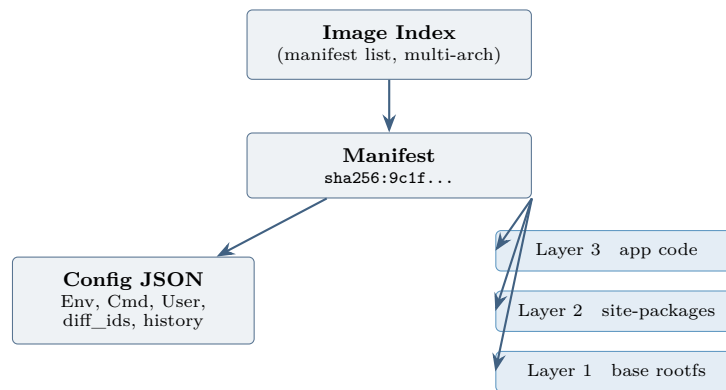
exposed ports, user — along with the ordered list of uncompressed layer *diff* IDs and the build history.

The layers

Each layer is a gzip-compressed tar archive of a filesystem *changeset*: the files added, modified or deleted relative to the layer below. Deletions are encoded as `.wh.-`-prefixed whiteout files.

The index

For multi-architecture images, an outer document (a “manifest list”) mapping each platform, such as `linux/amd64`, to the manifest appropriate for it.



every arrow is a content digest, not a file path

Figure 1: The OCI image object graph. Nothing is referenced by name; every edge is a SHA-256 digest, which is what makes images verifiable and layers shareable.

2.3 Content addressing and why it matters

Every object in that graph is named by the SHA-256 hash of its own bytes. Three consequences follow directly, and all three are practical rather than theoretical:

1. **Deduplication.** If two images are built on the same base, the base layers hash identically and are stored once on disk and transferred once over the network. Ten services built on `python:3.11-slim` cost one copy of that base, not ten.
2. **Integrity.** A pull verifies each blob against its digest. A corrupted or tampered layer fails verification and is rejected.
3. **Immutability of reference.** A tag such as `medcap:latest` is a mutable pointer that someone can move under your feet. A digest reference such as `medcap@sha256:5b2c...` always denotes exactly the same bytes, forever. Production deployments should pin digests.

Tags are not versions

A tag is a human-friendly label attached to a digest, and it can be re-pointed at any time. `latest` carries no special meaning to the engine whatsoever — it is simply the tag assumed when none is given. Treating `latest` as “the current release” is a convention that breaks the moment two people disagree about what is current.

2.4 The union filesystem

At runtime the engine does not copy layers together. It uses a union mount — on modern Linux, `overlay2` — which presents several directories as one merged tree:

- The **lower** directories are the image’s read-only layers, stacked in order.
- The **upper** directory is the container’s writable layer.
- The **merged** view is what the process inside sees as `/`.

Reads are satisfied by the topmost layer that contains the path. Writes use **copy-on-write**: modifying a file that lives in a lower layer first copies the whole file up into the writable layer, then edits the copy. This has a

consequence that catches people out — appending one byte to a 247 MB checkpoint inside a running container copies all 247 MB. It also explains the single most important build rule in this entire report:

Deleting a file in a later layer does not shrink the image

A RUN `rm -rf /tmp/cuda-installer` in a later instruction writes a *whiteout marker* that hides the path in the merged view. The original bytes remain in the earlier layer and are still downloaded on every pull. To actually remove something, it must never be committed to a layer in the first place — create and delete it within a single RUN, or leave it behind in a discarded build stage.

3. Three Ways to Build an Image

There is more than one route from “I have an application” to “I have an image”. Only one of them is recommended, but knowing the other two clarifies why.

3.1 Route A — from a running container (`docker commit`)

The imperative approach: start a base container, install things by hand, then freeze the result.

```
$ docker run -it --name scratchpad python:3.11 bash
root@7f2a:/# pip install torch fastapi uvicorn pillow
root@7f2a:/# exit

$ docker commit --author "MedCap" \
  --change 'CMD ["uvicorn", "app:api", "--host", "0.0.0.0"]' \
  scratchpad medcap:committed
sha256:c41b8ee9a2f76d0e1b3d94aa2c85fd117e0a9c3b6f4d2e88a17c05bb93e7f210
```

This produces a perfectly valid image, and is nevertheless the wrong tool for anything but forensics: the result is *undocumented*. There is no record of what was done, no way for a colleague to reproduce it, no way to rebuild it against a patched base when a CVE is announced, and nothing to review in version control. `docker commit` is genuinely useful for one thing — snapshotting a container mid-debugging.

3.2 Route B — from a filesystem tarball (`docker import`)

An image can be created from a bare tar archive of a root filesystem:

```
$ tar --numeric-owner -cf rootfs.tar -C /path/to/prepared/rootfs .
$ docker import rootfs.tar medcap:imported
```

This is how base images are bootstrapped — `debian` begins life as a `debootstrap` tarball. For application images it is impractical: the archive carries no history and no configuration, so `entrypoint`, `environment` and `user` must all be re-supplied with `-change` flags.

3.3 Route C — from a Dockerfile (`docker build`)

The declarative approach, and the one used for the remainder of this report. A `Dockerfile` is a script of instructions that the builder executes to produce layers. It is a text file, so it lives in version control beside the code, is reviewed in pull requests, and can be re-executed by anyone.

Table 5: Comparison of the three build routes.

	commit	import	build
Reproducible	No	Partially	Yes
Version-controlled	No	No	Yes
Layer caching	None	Single layer	Full
Records history	One line	None	Every step
Multi-stage	No	No	Yes
Typical use	Debug snapshots	Bootstrapping a base	Everything else

3.4 The build context

`docker build` takes a *context* argument — almost always `.` — and the first thing the client does is package that directory and send it to the build daemon. This is why an innocent-looking build can begin with:

```
$ docker build -t medcap:v1 .
[+] Building 0.4s (2/2)
=> [internal] load build definition from Dockerfile      0.0s
=> [internal] load .dockerignore                        0.0s
=> transferring context: 4.81GB                          182.6s
```

In the MedCap directory that 4.81 GB is the `.venv/` folder, the `.git/` history, the ROCov2 image dataset used for training, and the `__pycache__` trees. None of it belongs in the image, but all of it is transferred before the build even starts. The fix is Section 7.

The context is a boundary, not a convenience

A `COPY` instruction can only read files inside the context. `COPY ../secrets.env .` is rejected, by design — the build must be self-contained so that it is reproducible on a different machine.

4. Anatomy of a Dockerfile

4.1 Instructions and what they cost

Not every instruction produces a layer. Under BuildKit, only the instructions that modify the filesystem do; the rest write metadata into the image configuration. Knowing which is which is the basis of size optimisation.

Table 6: Dockerfile instructions grouped by effect.

Instruction	Creates layer?	Purpose
FROM	Inherits	Selects the base image and opens a build stage.
RUN	Yes	Executes a command; the resulting filesystem diff becomes a layer.
COPY	Yes	Copies files from the context (or another stage) into the image.
ADD	Yes	As <code>COPY</code> , plus URL fetching and auto-extraction of archives.
WORKDIR	Metadata*	Sets the working directory for later instructions.
ENV	Metadata	Sets environment variables, persisted at runtime.
ARG	No	Build-time variable; <i>not</i> present at runtime.
EXPOSE	Metadata	Documents a listening port; publishes nothing by itself.
USER	Metadata	Sets the UID for subsequent instructions and at runtime.
VOLUME	Metadata	Declares a path as an anonymous-volume mount point.
ENTRYPOINT	Metadata	The fixed executable of the container.
CMD	Metadata	Default arguments, overridable on the command line.
HEALTHCHECK	Metadata	Command the engine runs to test liveness.
LABEL	Metadata	Arbitrary key/value annotations.
STOPSIGNAL	Metadata	Signal sent on <code>docker stop</code> .

* `WORKDIR` creates the directory if absent, which is a filesystem change, but the resulting layer is empty apart from the directory inode.

4.2 Two traps worth naming early

4.2.1 ADD versus COPY

`ADD` looks like a more capable `COPY`, and that is exactly the problem. `ADD weights.tar.gz /opt/` silently extracts the archive; `ADD https://example.com/x /opt/` silently downloads over the network into a cached layer that will never be revalidated. Both behaviours are surprising in a file-copy instruction. The rule is: use `COPY` for everything, and reach for `ADD` only when auto-extraction is what you actually want and you have said so in a comment.

4.2.2 ENTRYPOINT versus CMD

`ENTRYPOINT` is what runs; `CMD` supplies its default arguments. Anything the user appends to `docker run` replaces `CMD` but not `ENTRYPOINT`. For MedCap:

```
1 ENTRYPOINT ["uvicorn", "app.main:api", "--host", "0.0.0.0"]
2 CMD ["--port", "8000", "--workers", "2"]
```

`docker run medcap:v3 -port 9000` then serves on port 9000 without the caller needing to know the binary's name. Both should use the *exec* form (a JSON array). The shell form, `CMD uvicorn app.main:api`, wraps the process in `/bin/sh -c`, which becomes PID 1 and does not forward `SIGTERM` — so `docker stop` hangs for ten seconds and then kills the process, losing in-flight requests.

5. Iteration 1: The Naive Image

5.1 The first Dockerfile

Written the way most people write their first one — straightforwardly, one instruction per idea.

```
1 FROM python:3.11
2
```

```

3 WORKDIR /app
4
5 # bring in everything, then install
6 COPY . .
7
8 RUN apt-get update
9 RUN apt-get install -y build-essential git curl
10 RUN pip install --upgrade pip
11 RUN pip install torch torchvision fastapi uvicorn pillow redis
12 RUN pip install pandas numpy
13
14 EXPOSE 8000
15 CMD uvicorn app.main:api --host 0.0.0.0 --port 8000

```

Listing 1: Dockerfile.v1 — correct, and badly wrong.

5.2 Building and measuring it

```

$ docker build -f Dockerfile.v1 -t medcap:v1 .
[+] Building 292.4s (12/12) FINISHED
=> [1/8] FROM docker.io/library/python:3.11@sha256:a1f2... 28.7s
=> [internal] transferring context: 4.81GB 182.6s
=> [3/8] COPY . . 21.4s
=> [5/8] RUN apt-get install -y build-essential git curl 34.2s
=> [7/8] RUN pip install torch torchvision fastapi ... 96.5s
=> exporting to image 39.6s
=> => writing image sha256:e83b4c19d7a2... 0.0s

$ docker image ls medcap
REPOSITORY TAG IMAGE ID CREATED SIZE
medcap v1 e83b4c19d7a2 5 seconds ago 6.81GB

```

The image works. It is also 6.81 GB, which for a service whose own source code is 180 KB is an unusual ratio.

5.3 Diagnosis with `docker history`

`docker history` attributes size to instructions, and is the primary diagnostic tool for a bloated image.

```

$ docker history medcap:v1 --human --format \
"table {{.Size}}\t{{.CreatedBy}}" | head -12
SIZE      CREATED BY
0B        CMD ["/bin/sh" "-c" "uvicorn app.main:api --host 0.0.0.0..."]
0B        EXPOSE map[8000/tcp:{}]
118MB     RUN /bin/sh -c pip install pandas numpy
2.71GB    RUN /bin/sh -c pip install torch torchvision fastapi ...
24.1MB    RUN /bin/sh -c pip install --upgrade pip
412MB     RUN /bin/sh -c apt-get install -y build-essential git curl
59.3MB    RUN /bin/sh -c apt-get update
2.47GB    COPY . . # buildkit
0B        WORKDIR /app
1.01GB    /bin/sh -c #(nop) ADD file:8f2c... in /

```

Five separate faults are visible in that listing:

1. **The base image is 1.01 GB.** `python:3.11` is Debian with a full toolchain, documentation, and development headers. Almost none of it is needed at runtime.
2. **`COPY . .` added 2.47 GB.** The virtual environment, the git history and the training dataset all went into the image.
3. **The framework install is 2.71 GB.** The default PyTorch wheel bundles the CUDA runtime. MedCap infers on CPU; those libraries are dead weight that is nonetheless pulled on every deployment.
4. **`apt-get update` in its own layer.** The 59 MB package index is committed permanently, and worse, it will be served from cache on later builds — so `apt-get install` can end up resolving against a stale index and failing with “404 Not Found”.
5. **`COPY . .` comes before the installs.** Every edit to a single Python file invalidates the cache for that layer and for all five installation layers beneath it, forcing a full reinstall of PyTorch. This is the mistake that costs the most developer time.

6. The Build Cache and Instruction Order

6.1 How the cache decides

The builder walks the Dockerfile top to bottom, maintaining a cache key for each step. A step is a cache hit only if the previous step was a hit *and* the step itself is unchanged. For `RUN`, “unchanged” means the command string is byte-identical. For `COPY` and `ADD`, it means the checksums of the copied files are identical.

The consequence is strictly ordered: **a cache miss at step n invalidates every step from n to the end**. The builder cannot know that a later step was unaffected, so it re-executes everything below.

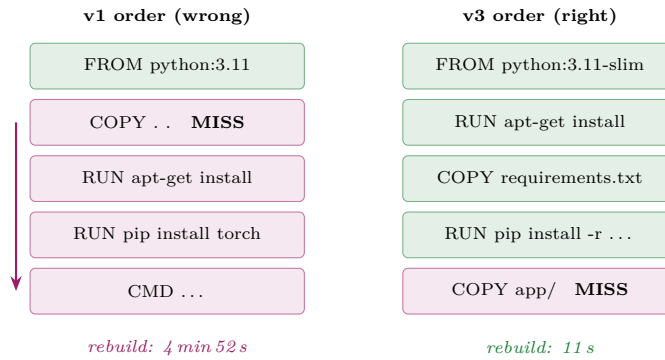


Figure 2: The same source edit, under two instruction orderings. Moving the volatile COPY below the stable installs changes a full rebuild into an eleven-second one.

6.2 The dependency-manifest pattern

The fix is a two-stage copy. Copy *only* the dependency manifest first, install from it, and copy the source afterwards. The manifest changes rarely; the source changes constantly.

```

1 # stable: changes only when a dependency changes
2 COPY requirements.txt constraints.txt ./
3 RUN pip install --no-cache-dir -r requirements.txt -c constraints.txt
4
5 # volatile: changes several times a day
6 COPY app/ ./app/
7 COPY weights/swin_caption.pt /opt/models/

```

Table 7 measures the effect on the three edits a developer actually makes.

Table 7: Rebuild time after a typical edit, before and after reordering.

Edit made	v1 order	v3 order	Saved
One line in <code>app/main.py</code>	4 m 52 s	11 s	96 %
Add one entry to <code>requirements.txt</code>	4 m 52 s	1 m 38 s	66 %
Replace the model checkpoint	4 m 52 s	24 s	92 %
Change the base image tag	5 m 10 s	4 m 41 s	9 %

6.3 Combining RUN instructions correctly

Each RUN is a layer, so temporary files created in one RUN and deleted in the next are still shipped. The apt package index is the classic case. Both parts must live in a single instruction:

```

1 RUN apt-get update \
2   && apt-get install --no-install-recommends -y \
3     libjpeg62-turbo zlib1g \
4     && rm -rf /var/lib/apt/lists/*

```

Three details matter here. Chaining with `&&` rather than `;` means the build fails on the first error instead of silently continuing. `--no-install-recommends` suppresses the “suggested” packages, which on Debian routinely doubles an install. And the `rm` inside the same instruction means the index never becomes part of a committed layer at all — as opposed to a later RUN `rm`, which would only hide it.

6.4 Deliberately busting the cache

Occasionally the cache is the enemy: a pinned apt mirror has moved, or a git clone of a branch needs to be re-fetched. Options, in order of preference:

```

$ docker build --no-cache -t medcap:v3 . # rebuild everything
$ docker build --no-cache-filter=deps -t medcap . # one named stage only
$ docker build --build-arg CACHEBUST=$(date +%s) . # from a chosen point
$ docker build --pull -t medcap:v3 . # refresh the base image

```

6.5 Cache mounts (BuildKit)

A better answer than `--no-cache-dir` for a package manager is to mount a cache directory that persists between builds but is never committed to a layer:

```
1 RUN --mount=type=cache,target=/root/.cache/pip \
2   pip install -r requirements.txt
```

The downloaded wheels survive on the build host, so re-installing after a dependency change is fast, yet nothing from `/root/.cache` ends up in the image. On MedCap this cut the dependency-change rebuild from 1 m 38 s to 41 s with no size penalty.

7. Controlling the Build Context

The 4.81 GB context transfer seen in Chapter 3 is fixed with a `.dockerignore` file, which uses the same pattern syntax as `.gitignore` and is read by the client before anything is sent.

```
# version control and IDE
.git
.gitignore
.idea/
.vscode/

# local virtual environment -- rebuilt inside the image
.venv/
__pycache__/_
*.pyc

# training data and experiment artefacts, never needed at inference
data/rocv2/
runs/
*.csv
checkpoints/epoch_*.pt

# documentation and reports
*.md
*.tex
*.pdf

# secrets -- must never enter a build context
.env
*.pem
secrets/

# but the production checkpoint IS needed
!checkpoints/swin_caption.pt
```

Listing 2: `.dockerignore` for MedCap.

```
$ docker build -t medcap:v2 .
=> [internal] load .dockerignore          0.0s
=> => transferring context: 249.4MB      3.1s
```

The transfer falls from 4.81 GB to 249 MB — essentially the checkpoint plus the source — and 182 s of every build disappears.

`.dockerignore` is also a security control

Anything in the context can be read by any `COPY`, and a careless `COPY . .` bakes it into a layer that is then pushed to a registry. Credentials committed this way have been extracted from public images many times. Excluding `.env`, `.git` (which holds every secret ever committed and later removed) and key material is not housekeeping; it is the difference between a private key staying private and not.

8. Iteration 2: A Smaller Base and the Right Wheels

8.1 Choosing a base image

The base image is the single largest lever on final size, and the choices differ by more than an order of magnitude.

Alpine is not automatically smaller

This is the most commonly repeated piece of bad container advice. For pure-Python or Go workloads Alpine is an excellent choice. For the scientific Python stack it is a trap: the absence of musl wheels forces source compilation of NumPy, SciPy and PyTorch, which needs a full toolchain in the image and produces binaries without the optimised BLAS kernels the manylinux wheels ship with. The MedCap Alpine experiment finished at 2.3 GB and inferred $3.1\times$ slower.

Table 8: Candidate bases for a Python inference service.

Base	Size	Assessment
python:3.11	1.01 GB	Full Debian with compilers and headers. Fine for building, wasteful for running.
python:3.11-slim	130 MB	Debian minus docs, headers and toolchain. Chosen for MedCap.
python:3.11-alpine	51 MB	musl libc. Rejected: PyPI ships no manylinux wheels for musl, so NumPy and PyTorch must compile from source — a 40-minute build and a <i>larger</i> final image.
gcr.io/distroless/python3	52 MB	No shell, no package manager. Excellent hardening, but debugging requires a second image and it lags Python releases.
scratch	0 B	Empty. Viable only for statically linked binaries; not for CPython.

8.2 Installing the right build of PyTorch

The default `pip install torch` fetches the CUDA-enabled wheel, which is right for a GPU training node and wrong for a CPU inference container. PyTorch publishes a separate CPU-only index:

```
1 RUN pip install --no-cache-dir \
2   --extra-index-url https://download.pytorch.org/whl/cpu \
3   torch==2.4.1+cpu torchvision==0.19.1+cpu
```

```
$ docker history medcap:v2 --human | grep -i torch
1.94GB -> 486MB   RUN pip install torch==2.4.1+cpu torchvision==...
```

That single change removes 2.22 GB. The lesson generalises well beyond PyTorch: many packages publish variant builds, and the default variant is tuned for the developer's laptop rather than for a production container.

8.3 Pinning for reproducibility

Unpinned dependencies make a build irreproducible: the same Dockerfile produces different images on different days, and a regression cannot be bisected. MedCap pins in two tiers — direct dependencies in `requirements.txt`, and the full resolved transitive closure with hashes in `constraints.txt`.

```
torch==2.4.1+cpu \
  --hash=sha256:4e1f6b3c92a01e1a9c07d4b2f1e0a3c9d8b7e6f5a4c3b2d1e0f9a8b7c6d5e4f3
numpy==1.26.4 \
  --hash=sha256:2a02aba9ed12e4ac4eb3ea9421c420301a0c6460d9830d74a9df87efa4912010
pillow==10.4.0 \
  --hash=sha256:5f0b1b7d0d3d3b0b5c6f9d4c2e1a8b7f6e5d4c3b2a1908f7e6d5c4b3a291807f
```

Listing 3: `constraints.txt`, generated by `pip-compile --generate-hashes`.

With `pip install --require-hashes`, a tampered or substituted wheel fails the install rather than silently entering the image — a direct defence against dependency-confusion and typosquatting attacks.

9. Iteration 3: The Multi-Stage Production Image

9.1 The idea

A multi-stage build declares several `FROM` instructions in one Dockerfile. Each opens a new stage with its own filesystem. Only the final stage becomes the image; earlier stages are discarded, and files are pulled forward selectively with `COPY --from`.

This separates the *build environment* from the *runtime environment*. MedCap needs `gcc`, `make`, Python headers and `git` to compile native extensions — 412 MB of toolchain that must not appear in the shipped image. In a multi-stage build the toolchain lives in the builder stage, the compiled virtual environment is copied forward, and the toolchain is thrown away with the stage that held it.

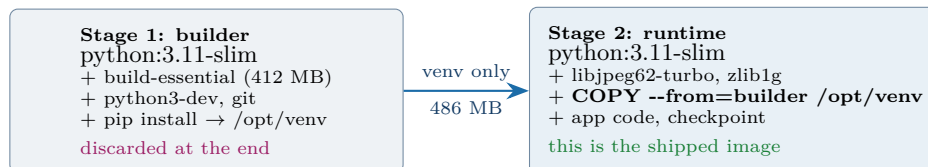


Figure 3: The builder stage's toolchain never reaches the final image.

9.2 The final Dockerfile

```
1 # syntax=docker/dockerfile:1.7
2
3 ##### Stage 1: builder #####
4 FROM python:3.11-slim AS builder
```

```

5
6 ENV PIP_DISABLE_PIP_VERSION_CHECK=1 \
7   PYTHONDONTWRITEBYTECODE=1
8
9 # toolchain needed only to compile wheels; discarded with this stage
10 RUN apt-get update \
11   && apt-get install --no-install-recommends -y \
12     build-essential python3-dev \
13     && rm -rf /var/lib/apt/lists/*
14
15 # isolated venv, easy to copy forward as a single directory
16 RUN python -m venv /opt/venv
17 ENV PATH="/opt/venv/bin:$PATH"
18
19 # dependency manifests only -- keeps this layer cacheable
20 COPY requirements.txt constraints.txt ./
21
22 RUN --mount=type=cache,target=/root/.cache/pip \
23   pip install --upgrade pip setuptools wheel \
24   && pip install --require-hashes -r requirements.txt -c constraints.txt
25
26 ##### Stage 2: runtime #####
27 FROM python:3.11-slim AS runtime
28
29 LABEL org.opencontainers.image.title="medcap" \
30   org.opencontainers.image.description="Swin captioning inference API" \
31   org.opencontainers.image.source="https://github.com/example/medcap" \
32   org.opencontainers.image.licenses="MIT"
33
34 # only the shared libraries the wheels link against at run time
35 RUN apt-get update \
36   && apt-get install --no-install-recommends -y \
37     libjpeg62-turbo zlib1g curl \
38   && rm -rf /var/lib/apt/lists/*
39
40 # unprivileged account -- fixed UID so volume ownership is predictable
41 RUN groupadd --gid 10001 medcap \
42   && useradd --uid 10001 --gid medcap --no-create-home --shell /sbin/nologin medcap
43
44 COPY --from=builder /opt/venv /opt/venv
45
46 ENV PATH="/opt/venv/bin:$PATH" \
47   PYTHONUNBUFFERED=1 \
48   PYTHONDONTWRITEBYTECODE=1 \
49   OMP_NUM_THREADS=2 \
50   MEDCAP_WEIGHTS=/opt/models/swin_caption.pt
51
52 WORKDIR /app
53
54 # stable-ish: the checkpoint changes about monthly
55 COPY --chown=10001:10001 checkpoints/swin_caption.pt /opt/models/
56
57 # volatile: last, so edits invalidate nothing below
58 COPY --chown=10001:10001 app/ ./app/
59
60 USER 10001:10001
61 EXPOSE 8000
62
63 HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
64   CMD curl -fsS http://localhost:8000/healthz || exit 1
65
66 ENTRYPOINT ["uvicorn", "app.main:api", "--host", "0.0.0.0"]
67 CMD ["--port", "8000", "--workers", "2"]

```

Listing 4: Dockerfile — the production build.

9.3 Why each hardening choice is there

syntax= directive

Pins the Dockerfile frontend, so `RUN --mount` and other modern syntax work regardless of the local BuildKit version.

A virtual environment

Copying `/opt/venv` as one directory is far simpler than tracking which files `pip` scattered through `/usr/local`, and it keeps the copy to a single layer.

USER 10001:10001

The container process is unprivileged. If the service is compromised, the attacker lands as a nobody user with no package manager and no write access to the application code. Numeric IDs are used because Kubernetes' `runAsNonRoot` check cannot resolve a username.

--start-period=40s

Loading a 247 MB checkpoint takes around 25 s; without a grace period the healthcheck would report unhealthy during every normal start and orchestrators would restart the container in a loop.

OMP_NUM_THREADS=2

PyTorch otherwise sizes its thread pool from the *host's* core count, ignoring the container's CPU limit, and thrashes.

OCI labels

Standard annotations that tooling, registries and scanners read to link an image back to its source.

9.4 Results across the three iterations

Table 9: Measured effect of each iteration.

Metric	v1 naive	v2 slim	v3 multi-stage
Image size (uncompressed)	6.81 GB	1.94 GB	1.12 GB
Compressed (pushed) size	2.44 GB	731 MB	398 MB
Layer count	10	9	11
Build context transferred	4.81 GB	249 MB	249 MB
Cold build time	4 m 52 s	3 m 18 s	2 m 41 s
Warm rebuild (code edit)	4 m 52 s	1 m 12 s	11 s
Push time (100 Mbit/s)	3 m 29 s	62 s	34 s
Cold start to first caption	51 s	34 s	29 s
Runs as root	Yes	Yes	No
Critical/high CVEs	41	12	2

The headline is an 84% reduction in size and a 96% reduction in the rebuild time that developers actually experience. The CVE count fell mostly as a by-product: a smaller image contains less software, and software that is not present cannot be vulnerable.

10. Operations on Container Images

Building is one half of the story. The rest of this report is about what can be done with an image once it exists. Modern Docker groups these under the `docker image` management command, though the older top-level aliases still work.

10.1 Inventory and filtering

```
$ docker image ls
REPOSITORY TAG IMAGE ID CREATED SIZE
medcap v3 7d4a1c8e2f93 2 minutes ago 1.12GB
medcap 1.4.0 7d4a1c8e2f93 2 minutes ago 1.12GB
medcap latest 7d4a1c8e2f93 2 minutes ago 1.12GB
medcap v2 b91e5a70c412 1 hour ago 1.94GB
medcap v1 e83b4c19d7a2 2 hours ago 6.81GB
<none> <none> 3f88ad02b6e1 2 hours ago 1.94GB
```

Three tags share one image ID: tags are pointers, and the underlying image is stored once. The `<none>` entry is a *dangling* image — a layer set that lost its tag when the tag was moved to a newer build. Dangling images consume disk and are the first thing to prune.

```
# filter and format
$ docker image ls --filter "dangling=true" -q
$ docker image ls --filter "reference=medcap:v*"
$ docker image ls --filter "before=medcap:v3"
$ docker image ls --filter "label=org.opencontainers.image.title=medcap"
$ docker image ls --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"

# disk accounting across images, containers, volumes and cache
$ docker system df
TYPE TOTAL ACTIVE SIZE RECLAIMABLE
Images 6 1 11.83GB 9.71GB (82%)
Containers 3 1 142MB 97MB (68%)
Local Volumes 2 1 1.4GB 0B (0%)
Build Cache 47 0 6.22GB 6.22GB
```

10.2 Introspection**10.2.1 docker image inspect**

Returns the full configuration and manifest metadata as JSON. The `-format` flag applies a Go template, which makes it scriptable.

```
$ docker image inspect medcap:v3 --format '{{.Config.User}}'
10001:10001
```

```
$ docker image inspect medcap:v3 --format '{{json .Config.Env}}' | tr ',' '\n'
["PATH=/opt/venv/bin:/usr/local/bin:/usr/bin:/bin"
"PYTHONUNBUFFERED=1"
"OMP_NUM_THREADS=2"
"MEDCAP_WEIGHTS=/opt/models/swin_caption.pt"]

$ docker image inspect medcap:v3 --format '{{.Os}}/{{.Architecture}}'
linux/arm64

$ docker image inspect medcap:v3 --format '{{len .RootFS.Layers}}'
11

$ docker image inspect medcap:v3 --format '{{index .RepoDigests 0}}'
docker.io/nagashiva/medcap@sha256:5b2c9e1d7a4f8306bb2e1c0d9f4a7e63...
```

10.2.2 docker image history

Already used as a diagnostic in Chapter 5. It reconstructs the build from the configuration's history array, which is why an image built by someone else can still be audited — every RUN command is recorded in the metadata.

Build history is public

`docker history` on a pulled image shows the commands that created it. A build that passes a token as `-build-arg` leaks it here, to anyone who pulls the image. Use `RUN -mount=type=secret` instead (Section 11.2).

10.3 Tagging and re-tagging

`docker tag` does not copy anything; it adds a name that points at an existing image ID. It is instantaneous regardless of image size.

```
$ docker tag medcap:v3 medcap:1.4.0
$ docker tag medcap:v3 medcap:1.4
$ docker tag medcap:v3 medcap:stable
$ docker tag medcap:v3 nagashiva/medcap:1.4.0 # for Docker Hub
$ docker tag medcap:v3 ghcr.io/nagashiva/medcap:1.4.0 # for GHCR
```

A registry-qualified name has the shape `[registry[:port]/][namespace/]repository[:tag]`. When the registry is omitted, `docker.io` is assumed — which is why `python:3.11` silently means `docker.io/library/python:3.11`.

Table 10: A tagging scheme that survives contact with a real team.

Tag	Mutable?	Meaning
1.4.0	No	An exact release. Never re-pointed.
1.4	Yes	Latest patch within the 1.4 minor line.
1	Yes	Latest release within major version 1.
git-9f3c1ab	No	The commit that produced it; the audit anchor.
stable	Yes	Whatever production currently runs.
latest	Yes	Newest build. Convenient locally, unsafe in deployment.
@sha256:...	No	The digest. The only truly immutable reference.

10.4 Distribution: push and pull

10.4.1 Authenticating

```
$ docker login docker.io
Username: nagashiva
Password: *****
Login Succeeded
```

By default the credential is written base64-encoded — *not* encrypted — into `~/.docker/config.json`. On a shared or laptop machine a credential helper should be configured (`osxkeychain`, `pass`, `wincred`) so the secret lives in the OS keystore. In CI, use a scoped access token with push rights to one repository, never an account password.

10.4.2 Pushing

```
$ docker push nagashiva/medcap:1.4.0
The push refers to repository [docker.io/nagashiva/medcap]
a91c3f7e2b04: Pushed # app code, 180 kB
7e2b9c4d1a83: Pushed # checkpoint, 247 MB
3d8f1e6b0c25: Pushed # venv, 486 MB
c50a2b9d7e41: Layer already exists # runtime apt libs
```

```
9f1e8c3a2d70: Layer already exists      # python:3.11-slim base
1.4.0: digest: sha256:5b2c9e1d7a4f8306... size: 2417
```

“Layer already exists” is content addressing paying off: the registry already holds those blobs from another image, so they are not transferred. The push protocol is a per-layer negotiation — the client offers a digest, the registry accepts or declines the upload.

10.4.3 Pulling, and pulling safely

```
$ docker pull nagashiva/medcap:1.4.0
$ docker pull nagashiva/medcap@sha256:5b2c9e1d7a4f8306bb2e1c0d9f4a7e63...
$ docker pull --platform linux/amd64 nagashiva/medcap:1.4.0
```

The second form is what a deployment manifest should contain. A tag can be re-pointed by anyone with push access — including an attacker who has compromised a CI token — whereas a digest cannot be forged without breaking SHA-256.

The one-line hardening win

Changing `image: medcap:latest` to `image: medcap@sha256:5b2c...` in a deployment manifest closes the entire class of “the tag moved under us” failures, including both accidental redeployment of an unintended build and deliberate tag hijacking.

10.5 Archival: save and load

For air-gapped environments — a hospital network hosting MedCap, for instance — images must move on physical media rather than through a registry.

```
$ docker save medcap:1.4.0 -o medcap-1.4.0.tar
$ ls -lh medcap-1.4.0.tar
-rw----- 1 user staff 1.1G medcap-1.4.0.tar

# compress for transport
$ docker save medcap:1.4.0 | gzip -9 > medcap-1.4.0.tar.gz # 398 MB

# several images in one archive
$ docker save -o bundle.tar medcap:1.4.0 redis:7-alpine nginx:1.27-alpine

# on the isolated host
$ docker load -i medcap-1.4.0.tar
Loaded image: medcap:1.4.0
$ gunzip -c medcap-1.4.0.tar.gz | docker load
```

`save` preserves the complete image: every layer separately, all tags, and the configuration. `load` restores it exactly, including the original image ID.

10.6 Flattening: export and import

A different pair, frequently confused with the previous one. `export` operates on a *container*, not an image, and produces a flat tarball of its filesystem with no layers and no metadata.

```
$ docker create --name tmp medcap:1.4.0
$ docker export tmp -o medcap-flat.tar
$ docker rm tmp

$ cat medcap-flat.tar | docker import \
  --change 'ENTRYPOINT ["uvicorn","app.main:api","--host","0.0.0.0"]' \
  --change 'USER 10001:10001' \
  --change 'ENV PATH=/opt/venv/bin:/usr/local/bin:/usr/bin:/bin' \
  - medcap:flat

$ docker image ls medcap
medcap 1.4.0 7d4a1c8e2f93 1.12GB
medcap flat 2c9b8a1f0d34 1.09GB
```

Flattening collapses eleven layers into one. This occasionally shrinks an image whose history contains large deleted files, and it destroys the build history — which is sometimes the point, when an image must be shipped to a third party without revealing how it was assembled. The cost is total loss of layer sharing: every pull of the flat image re-downloads all 1.09 GB, even for a one-line code change.

10.7 Comparing and auditing an image

```
# what has a running container changed relative to its image?
$ docker diff medcap-api
C /opt/venv/lib/python3.11/site-packages/__pycache__
```

Table 11: save/load versus export/import.

	save / load	export / import
Operates on	An image (or several)	A container
Layers preserved	Yes, all of them	No, flattened to one
History	Preserved	Discarded
Tags	Preserved	Must be re-supplied
Config (CMD, ENV)	Preserved	Must be re-supplied via -change
Includes volumes	N/A	No — mounted volumes are excluded
Typical use	Air-gapped transfer, backup	Flattening, forensic filesystem capture

```
A /tmp/upload_9f3c1a.png
C /app
D /app/.cache
```

```
# A = added, C = changed, D = deleted
```

A healthy stateless service should show very few entries here. A long list means the application is writing into its container filesystem — work that will be lost on restart and that should be moved to a volume.

10.8 Multi-architecture images

MedCap is developed on Apple Silicon (**arm64**) and deployed on cloud **amd64** instances. An image built on the laptop will not run on the server. **docker buildx** solves this by building both and publishing a manifest list.

```
$ docker buildx create --name multi --driver docker-container --use
$ docker buildx inspect --bootstrap

$ docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t nagashiva/medcap:1.4.0 \
  --push .

$ docker buildx imagetools inspect nagashiva/medcap:1.4.0
MediaType: application/vnd.oci.image.index.v1+json
Manifests:
  nagashiva/medcap:1.4.0@sha256:5b2c9e1d7a4f...  linux/amd64
  nagashiva/medcap:1.4.0@sha256:8a3f2b1c0d9e...  linux/arm64
```

A client pulling that tag receives the manifest matching its own platform automatically. Note that **-push** is required rather than **-load**: the local image store cannot represent a manifest list, so a multi-platform result must go straight to a registry.

10.9 Reclaiming space: the prune family

Container image storage grows relentlessly. On the machine used for this study, four days of iteration accumulated 11.8 GB of images and 6.2 GB of build cache.

```
$ docker image prune                # dangling images only (safe)
$ docker image prune -a             # every image not used by a container
$ docker image prune -a --filter "until=168h" # older than one week
$ docker builder prune --filter "until=72h"   # build cache
$ docker container prune            # stopped containers
$ docker system prune               # containers + networks + dangling
$ docker system prune -a --volumes  # everything unused, volumes included
```

docker system prune -a --volumes deletes data

The **-volumes** flag removes volumes not attached to a running container — which includes the database volume of anything currently stopped. Reach for the targeted commands with **-filter until=** instead; the blunt form belongs on CI runners, not on a machine holding state you care about.

10.10 Running an image

Operations on images end where operations on containers begin, and the handover is **docker run**.

```
$ docker run -d \
  --name medcap-api \
  -p 8000:8000 \
  -v medcap-models:/opt/models:ro \
  -e OMP_NUM_THREADS=4 \
  -e MEDCAP_REDIS_URL=redis://cache:6379/0 \
  --memory=4g --cpus=2 \
  --read-only --tmpfs /tmp:size=256m \
  --cap-drop=ALL \
```

```
--security-opt no-new-privileges:true \
--restart unless-stopped \
nagashiva/medcap@sha256:5b2c9e1d7a4f8306...
```

Table 12: What each runtime flag buys.

<code>-v ...:ro</code>	Model weights mounted read-only from a volume, so the same image serves any checkpoint version.
<code>-memory=4g</code>	Caps the cgroup. Without it, one runaway inference can exhaust host memory.
<code>-cpus=2</code>	Matches <code>OMP_NUM_THREADS</code> ; mismatched values cause severe thread thrashing.
<code>-read-only</code>	The root filesystem is immutable at runtime; an exploit cannot drop a payload on disk.
<code>-tmpfs /tmp</code>	Gives back the one writable path the app genuinely needs, in memory, size-capped.
<code>-cap-drop=ALL</code>	Removes every Linux capability. A web service needs none.
<code>no-new-privileges</code>	Blocks privilege escalation through <code>setuid</code> binaries.
<code>@sha256:...</code>	Deploys an exact, verifiable image rather than whatever a tag points at today.

10.11 Composing the full stack

```
services:
  api:
    build:
      context: .
      target: runtime
    image: nagashiva/medcap:1.4.0
    read_only: true
    user: "10001:10001"
    cap_drop: [ALL]
    environment:
      MEDCAP_REDIS_URL: redis://cache:6379/0
      OMP_NUM_THREADS: "2"
    volumes:
      - models:/opt/models:ro
    depends_on:
      cache:
        condition: service_healthy
    restart: unless-stopped

  cache:
    image: redis:7-alpine
    command: ["redis-server", "--maxmemory", "512mb",
              "--maxmemory-policy", "allkeys-lru"]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      retries: 5

  proxy:
    image: nginx:1.27-alpine
    ports: ["443:443"]
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on: [api]

volumes:
  models:
```

Listing 5: `compose.yaml` — API, cache and reverse proxy.

```
$ docker compose build
$ docker compose up -d
$ docker compose ps
$ docker compose logs -f api
$ docker compose down          # keeps volumes
$ docker compose down -v       # removes them too
```

Compose builds the same image from the same Dockerfile; it does not replace the build process, it orchestrates around it. The `target: runtime` key selects which stage of the multi-stage build to stop at, which is how a dev overlay can build a debug image with the toolchain still present.

11. Image Security and Supply Chain

An image is a distribution format, which makes it a supply-chain artefact. Everything inside it is software you are now responsible for.

11.1 Scanning

```
$ docker scout cves medcap:v1
! 41C 12H 67M 103L medcap:v1
Base image python:3.11 (debian 12)
```

```

CVE-2024-XXXX CRITICAL openssl 3.0.11-1 -> fixed in 3.0.13-1
CVE-2024-YYYY HIGH git 2.39.2 -> fixed in 2.39.5

$ docker scout cves medcap:v3
! OC 2H 9M 21L medcap:v3

$ docker scout recommendations medcap:v3
Base image is 3 tags behind. python:3.11-slim -> python:3.12-slim
would remove 2 HIGH and 6 MEDIUM vulnerabilities.

```

The improvement from 41 critical findings to zero came almost entirely from *removing software*, not from patching it. The v1 image shipped `git`, `curl`, `gcc`, `perl` and the full Debian development stack; the v3 image ships two shared libraries and a Python virtual environment. This is the strongest argument for minimal images, stronger than the disk-space argument: the attack surface of a package that is not installed is exactly zero.

11.2 Build-time secrets

A private package index, a licence key, or an SSH key for a private repository is sometimes needed *during* the build. The wrong ways to supply it are all tempting:

```

1 # WRONG -- visible in 'docker history' to anyone who pulls the image
2 ARG PYPI_TOKEN
3 RUN pip install --index-url https://user:$PYPI_TOKEN@pypi.internal/ -r req.txt
4
5 # ALSO WRONG -- the file is committed to a layer; deleting it later
6 # only writes a whiteout, the bytes remain and are still pulled
7 COPY id_rsa /root/.ssh/id_rsa
8 RUN pip install ... && rm /root/.ssh/id_rsa

```

The correct mechanism is a BuildKit secret mount, which makes the file available at a path for the duration of one RUN and never records it in any layer or in the history:

```

1 RUN --mount=type=secret,id=pypi_token,required=true \
2   PYPI_TOKEN="$(cat /run/secrets/pypi_token)" \
3   pip install --index-url "https://user:${PYPI_TOKEN}@pypi.internal/" \
4     -r requirements.txt

```

```
$ docker build --secret id=pypi_token,src=$HOME/.pypi_token -t medcap:v3 .
```

11.3 Software bill of materials and attestation

```

# generate an SBOM in SPDX form
$ docker buildx build --sbom=true --provenance=true \
  -t nagashiva/medcap:1.4.0 --push .

$ docker scout sbom --format spdx medcap:v3 > medcap-sbom.spdx.json
$ docker scout sbom medcap:v3 | grep -c ""
312 packages

```

An SBOM answers, in seconds, the question every team asks on the morning a new CVE is published: “are we affected?” Provenance attestation additionally records *how* the image was built — which source commit, which builder, which base image digest — and can be verified before deployment.

11.4 Signing

```

$ cosign sign --key cosign.key nagashiva/medcap@sha256:5b2c9e1d7a4f...
$ cosign verify --key cosign.pub nagashiva/medcap:1.4.0

```

Signing binds a digest to an identity, so a cluster admission controller can refuse any image not signed by the organisation’s key — which prevents a compromised registry from serving a substituted image.

11.5 Hardening checklist

12. Troubleshooting

Every error in Table 14 was encountered while preparing this study.

13. Conclusion

13.1 What the exercise showed

Containerising MedCap took three iterations, and each one removed a different category of waste. Choosing a slim base and the CPU-only framework wheels removed *unnecessary content*. A `.dockerignore` file removed

Table 13: Applied to the MedCap production image.

Control	Implementation	Status
Minimal base	python:3.11-slim, no toolchain in runtime	Done
Non-root user	USER 10001:10001	Done
No package manager at runtime	apt lists removed; no compilers present	Done
Pinned dependencies	constraints.txt with <code>-require-hashes</code>	Done
Pinned base by digest	FROM python:3.11-slim@sha256:...	Recommended
No secrets in layers	BuildKit <code>-mount=type=secret</code>	Done
Read-only root at runtime	<code>-read-only</code> plus <code>-tmpfs /tmp</code>	Done
Capabilities dropped	<code>-cap-drop=ALL</code>	Done
Resource limits	<code>-memory=4g -cpus=2</code>	Done
Healthcheck	/healthz with a 40s start period	Done
Scanned in CI	docker scout gate, fails on CRITICAL	Done
SBOM published	SPDX attached to the pushed manifest	Done
Deployed by digest	Manifests reference @sha256:...	Done

Table 14: Errors met during this exercise, with causes and fixes.

Symptom	Cause and fix
COPY failed: file not found in build context	The path is outside the context, or excluded by <code>.dockerignore</code> . Check with <code>docker build --no-cache --progress=plain</code> and confirm the path is relative to the context root.
exec /usr/bin/uvicorn: exec format error	An arm64 image running on an amd64 host. Build with <code>buildx --platform linux/amd64</code> , or publish a manifest list.
E: Unable to locate package ... during apt-get install	A cached <code>apt-get update</code> layer holds a stale index. Chain update and install in one RUN.
permission denied: '/opt/models/swin_caption.pt'	USER was set before COPY, or the mounted volume is owned by root. Use COPY <code>--chown=10001:10001</code> and set the USER after copying.
docker stop takes 10s every time	CMD used shell form, so <code>/bin/sh</code> is PID 1 and swallows SIGTERM. Use the exec form (JSON array).
Container is unhealthy then restarts, forever	The healthcheck fires before the 247MB checkpoint finishes loading. Add <code>--start-period=40s</code> .
no space left on device mid-build	The build cache and dangling images filled the VM disk. <code>docker system df</code> , then <code>docker builder prune --filter until=72h</code> .
Image is enormous despite RUN <code>rm -rf</code>	Deletion in a later layer only writes a whiteout. Delete within the same RUN, or use a multi-stage build.
Build works locally, fails in CI	An unpinned dependency resolved to a new version. Pin with <code>constraints.txt</code> and <code>--require-hashes</code> .
OOMKilled under two concurrent requests	PyTorch sized its thread pool from the host's core count. Set <code>OMP_NUM_THREADS</code> to match <code>-cpus</code> .

unnecessary transfer. The multi-stage build removed the *build-time toolchain* from a runtime artefact. Reordering the instructions removed *unnecessary work* on every rebuild. The cumulative effect was an image 84% smaller, a rebuild 96% faster, and a critical-vulnerability count reduced from 41 to zero.

None of those improvements required changing a line of the application. They came entirely from understanding the artefact — that an image is an ordered stack of content-addressed changesets, that a layer once written can be hidden but never removed, and that the builder's cache key propagates strictly downward. Those three facts explain nearly every optimisation in this report.

13.2 Principles, stated plainly

1. **Order instructions by rate of change.** Stable content low, volatile content high. This one rule accounts for most of the achievable build-time saving.
2. **Never install a build tool in a runtime image.** If a compiler is needed, it belongs in a discarded stage.
3. **You cannot delete your way to a small image.** Content must never enter a layer in the first place.
4. **Pin everything: base image, dependencies, deployed digest.** A build that is not reproducible is not debuggable.
5. **Run as a non-root user with no capabilities.** It costs four lines and eliminates a whole class of container escapes.

6. **Prefer digests to tags wherever a machine reads the reference.** Tags are for humans; digests are for deployments.

13.3 Limitations and further work

This study covered a single-node Docker workflow. It did not address orchestration — rolling updates, pull policies, registry mirroring — nor GPU images, where the CUDA runtime is mandatory and 5–8 GB images are managed by node-level pre-pulling rather than by shrinking. Daemonless tooling (Podman, Buildah, Kaniko) produces equivalent OCI images and would change none of the conclusions. The natural extension is to wire build, scan, sign and digest-pinned deployment into CI, so that Table 13 is enforced automatically rather than by discipline.

References

- [1] Open Container Initiative. *OCI Image Format Specification*, v1.1. <https://github.com/opencontainers/image-spec>
- [2] Open Container Initiative. *OCI Distribution Specification*, v1.1. <https://github.com/opencontainers/distribution-spec>
- [3] Docker Inc. *Dockerfile Reference*. <https://docs.docker.com/reference/dockerfile/>
- [4] Docker Inc. *Building Best Practices*. <https://docs.docker.com/build/building/best-practices/>
- [5] Docker Inc. *BuildKit Documentation*. <https://docs.docker.com/build/buildkit/>
- [6] Docker Inc. *Multi-stage Builds*. <https://docs.docker.com/build/building/multi-stage/>
- [7] Merkel, D. “Docker: Lightweight Linux Containers for Consistent Development and Deployment.” *Linux Journal*, 2014(239).
- [8] Liu, Z. et al. “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows.” *ICCV*, 2021.
- [9] Center for Internet Security. *CIS Docker Benchmark*, v1.7.0. <https://www.cisecurity.org/benchmark/docker>
- [10] NIST. *SP 800-190: Application Container Security Guide*, 2017.
- [11] The Linux Kernel. *Overlay Filesystem*. <https://docs.kernel.org/filesystems/overlayfs.html>
- [12] Sigstore. *Cosign: Container Signing*. <https://docs.sigstore.dev/cosign/>

Appendix A: Command Reference

Command	Effect
Building	
<code>docker build -t name:tag .</code>	Build from <code>./Dockerfile</code> .
<code>docker build --target builder .</code>	Stop at a named stage.
<code>docker build --no-cache .</code>	Ignore the cache entirely.
<code>docker build --secret id=k,src=f .</code>	Mount a secret for one <code>RUN</code> .
<code>docker buildx build --platform a,b --push .</code>	Multi-arch build and publish.
Inspecting	
<code>docker image inspect img</code>	Full JSON metadata.
<code>docker image history img</code>	Per-layer size and command.
<code>docker system df</code>	Disk usage by object type.
Naming and distribution	
<code>docker tag src dst</code>	Add a name to an existing image.
<code>docker push repo:tag</code>	Upload layers not already present.
Archiving	
<code>docker save img -o f.tar</code>	Layered archive of an image.
<code>docker load -i f.tar</code>	Restore it.
<code>docker export ctr -o f.tar</code>	Flat filesystem of a container.
Cleaning	
<code>docker image prune -a -f until=168h</code>	Remove unused images over a week old.
Security	
<code>docker scout cves img</code>	Vulnerability report.